

GitOps Self-Service Deployment Platform

A Complete Plain-English Explanation of the Project,
Its Components, Its Workflow and Its Architecture

*Declarative delivery on Argo CD and Flux, with Git as the single source of truth
for Kubernetes application and infrastructure state*

Outcomes: deployment lead time reduced by 55% | deployment success rate above 99%

Prepared 27 August 2026

Contents

1. Executive Summary (In Simple Terms).....	7
1.1 The Project in One Paragraph	7
1.2 The Resume Line, Decoded Phrase by Phrase	7
1.3 The Problem This Solved	8
1.4 What Was Actually Built.....	8
1.5 The Results and What They Mean	9
2. Background Concepts, Explained From Zero	10
2.1 What a Deployment Actually Is.....	10
2.2 What Kubernetes Is (In One Minute)	10
2.3 Declarative vs Imperative — the Central Distinction	10
2.4 Desired State vs Live State	11
2.5 What GitOps Means.....	11
2.6 Push vs Pull — and Why Pull Is More Secure	11
2.7 Why "Self-Service" Matters	12
3. Every Component, Explained End to End.....	13
3.1 The Self-Service Platform Layer	13
What it is.....	13
Why it is needed	13
What was configured.....	13
Its role end to end.....	13
Analogy	13
3.2 Git as the Single Source of Truth.....	14
What it is.....	14
Why it is needed	14
What was configured — the repository layout.....	14
Why source code and configuration are kept apart	15
Analogy	15
3.3 The CI Pipeline (Build and Test)	15
What it is.....	15
The stages, in order	15
Why immutable tags matter so much	16

Analogy	16
3.4 The Container Registry.....	16
3.5 Helm.....	16
What it is	16
Its parts	16
Why it is needed	17
How it is used here	17
Analogy	17
3.6 Kustomize.....	17
What it is	17
Why both Helm and Kustomize are used.....	18
Analogy	18
3.7 Argo CD	18
What it is	19
Its concepts	19
What it contributes to the results.....	19
Analogy	20
3.8 Flux.....	20
What it is	20
Its controllers, one by one	20
Its objects.....	20
Analogy	21
3.9 Why Both Argo CD and Flux Were Used	21
3.10 Automated Reconciliation	21
What it is	21
The loop, step by step.....	22
What triggers a pass	22
Why this is the biggest single contributor to the results	22
3.11 Drift Detection	22
What drift is	22
Where drift comes from	22
Auto-heal versus alert-only.....	23
Why drift detection is worth so much	23

Analogy	23
3.12 Environment Promotion	23
What it is	23
The essential rule	24
What actually happens at each step	24
Why promotion through Git is better than promotion through a pipeline	24
3.13 One-Click Rollback.....	24
What it is	25
The two routes, and when each is used.....	25
Why it is fast and safe	25
Why fast rollback increases speed overall	25
3.14 Secrets Management	25
3.15 Policy, Validation and Guardrails	26
3.16 Observability, Notifications and Metrics.....	26
3.17 Access Control and the Security Model	27
4. The End-to-End Workflow, Stage by Stage	28
4.1 The Whole Workflow at a Glance	28
4.2 Stage 0 — Onboarding a New Service (Once)	28
4.3 Stage 1 — The Developer Writes Code	29
4.4 Stage 2 — CI Builds and Proves the Artifact.....	29
4.5 Stage 3 — The Automated Configuration Pull Request	29
4.6 Stage 4 — Automated Checks on the Configuration Change.....	30
4.7 Stage 5 — Review and Merge	30
4.8 Stage 6 — The Controller Learns About the Commit	30
4.9 Stage 7 — Rendering the Manifests.....	31
4.10 Stage 8 — The Diff	31
4.11 Stage 9 — Applying the Change	31
4.12 Stage 10 — Kubernetes Performs the Rolling Update	32
4.13 Stage 11 — Verification and Reporting.....	32
4.14 Stage 12 — Promotion Onward	32
4.15 Stage 13 — Continuous Reconciliation Forever	33
4.16 Stage 14 — Rollback.....	33
4.17 A Worked Timeline: Before and After.....	33

4.18 What Goes Wrong at Each Stage, and What Happens Then.....	34
5. The Architecture Diagram, Explained in Full.....	36
5.1 How to Read These Diagrams	36
5.2 Figure 1 — The Whole System in One Picture	36
Figure 1, box by box	38
Figure 1, arrow by arrow.....	38
5.3 Figures 2A and 2B — Architecture Part A: From Person to Git.....	39
Layer 1 — People who use the platform.....	39
Layer 2 — The self-service platform	39
Layer 3 — The CI pipeline	41
Layer 4 — Git, the single source of truth	41
5.4 Figure 3 — Architecture Part B: From Git to Running Software	43
Layer 5 — The GitOps controllers	43
Layer 6 — The rendering engine	44
Layer 7 — The Kubernetes clusters.....	44
5.5 Figure 4 — The Reconciliation and Drift-Detection Loop	46
Each step of the loop	46
5.6 Figure 5 — Environment Promotion	49
5.7 Figure 6 — Supporting Services	49
5.8 Figure 7 — One-Click Rollback	51
5.9 Every Arrow in the Architecture, in One Table	51
6. How the Results Were Achieved and Measured.....	53
6.1 Where the 55% Lead-Time Reduction Came From.....	53
6.2 Where the Above-99% Success Rate Came From	53
6.3 The Metrics That Are Tracked	54
7. Design Decisions and Honest Trade-Offs	55
7.1 What This Architecture Does Not Solve.....	55
8. Glossary.....	56
9. Questions People Usually Ask About This Project	59
Why not just use a normal CI/CD pipeline to deploy?	59
What stops someone bypassing all of this with kubectl?	59
Isn't running both Argo CD and Flux redundant?	59
What happens if Git is unavailable?.....	59

How do you deploy something urgently at 3am?	59
What was the hardest part?	59

If the contents list above appears empty or out of date, open it in Word and press Ctrl+A then F9 to refresh the field.

How to read this document

Sections 1 and 2 give the summary and the background ideas. Section 3 explains every component. Section 4 walks the whole workflow stage by stage. Section 5 contains the architecture diagrams with a full explanation of every box and every arrow. Sections 6 to 9 cover measurement, trade-offs, a glossary and common questions. No prior Kubernetes knowledge is assumed anywhere.

1. Executive Summary (In Simple Terms)

1.1 The Project in One Paragraph

This project replaced a slow, manual, error-prone way of putting software into production with an automatic one. Before, an engineer had to log into servers and run commands by hand to release software. After, the entire description of what should be running is written down as files in **Git** (the same place code is stored). Software running inside the Kubernetes clusters constantly reads those files and makes the real system match them. A developer no longer "does a deployment" at all. A developer simply changes a file, gets it reviewed, and merges it. Everything after that happens by itself. The result was that changes reached production **55% faster**, and **more than 99% of deployments succeeded** instead of regularly failing.

The single most important idea in this whole document

Git is the **single source of truth**. If it is not written in Git, it does not exist. If someone changes the live system without changing Git, the system will notice and undo that change.

Everything else in this document — Argo CD, Flux, Helm, Kustomize, reconciliation, drift detection, promotion, rollback — is just machinery that exists to make that one idea true, reliably, at scale.

1.2 The Resume Line, Decoded Phrase by Phrase

The original sentence is written in industry shorthand. Here is exactly what each phrase means in ordinary language.

Phrase used	What it actually means
Cut deployment lead time 55%	The time between "a developer finishes a change" and "customers are using that change" dropped by more than half. If it used to take about 4 hours, it now takes under 2.
Raised deployment success rate above 99%	Out of every 100 attempted releases, fewer than 1 fails or has to be abandoned. Before, failures were common enough to be a normal part of the week.
Defining the architecture for a self-service deployment platform	I designed the overall shape of the system: which pieces exist, what each piece is responsible for, how they talk to each other, and what rules they enforce. "Self-service" means developers can deploy their own software without filing a ticket and waiting for a platform engineer.
Shipping declarative GitOps delivery	"Declarative" = you write down the end result you want, not the steps to get there. "GitOps" = that written-down description lives in Git, and automation makes reality match it. "Delivery" = getting software into environments. "Shipping" = I actually built and released it, not just designed it.
On Argo CD and Flux	The two specific open-source tools that do the automatic matching. They run inside the Kubernetes cluster and continuously compare Git against reality.

Phrase used	What it actually means
With Git as source of truth for Kubernetes application and infrastructure state	Git holds the description of both the applications (the services customers use) and the infrastructure around them (networking, monitoring, permissions, namespaces).
Automated reconciliation	The system repeatedly asks "does reality match Git?" and fixes reality if it does not. Nobody has to trigger it.
Drift detection	Noticing when the live system has quietly stopped matching Git — usually because someone made a manual change or another tool interfered.
Helm and Kustomize configuration	Two ways of writing Kubernetes configuration so that one description can serve several environments with small differences, instead of copying and pasting the same files three times.
Environment promotion	Moving a specific, tested version of software forward through development, then staging, then production — in a controlled, reviewable way.
One-click rollback	If a release goes wrong, returning to the previous working version is a single action that takes a couple of minutes, not a stressful manual recovery.

1.3 The Problem This Solved

Before this platform existed, releasing software looked roughly like this:

- A developer finished a change and then **waited** for a platform engineer to be available to deploy it.
- The platform engineer ran commands by hand, or ran a script, against the cluster. Different people did it slightly differently.
- Configuration lived in several places: some in a wiki, some in a script, some only in one engineer's memory, and some only in the live cluster itself.
- Nobody could say with certainty what was actually running in production, because the live system had been hand-edited many times since it was set up.
- When a release went wrong, recovery meant improvising under pressure, because there was no reliable record of the last known-good state.
- Environments (development, staging, production) drifted apart over time, so "it worked in staging" stopped being a meaningful statement.
- Deployment credentials had to be handed out to pipelines and to people, which made the security team uncomfortable.

Every one of those problems has the same root cause: **there was no single, trustworthy, written-down description of what the system should be**. This project fixed the root cause rather than the individual symptoms.

1.4 What Was Actually Built

Six things were designed and delivered, and they work together as one platform.

#	What was built	What it gives the organisation
1	A self-service platform layer (portal, CLI and golden-path templates)	A developer can create a new service or release an existing one without knowing Kubernetes and without asking anyone for help.
2	Git as the single source of truth, with structured repositories	One place that always answers the question "what is supposed to be running?" — with full history and review on every change.
3	GitOps controllers (Argo CD and Flux) running inside the clusters	Automatic, continuous delivery that pulls changes rather than being pushed to, which is both faster and more secure.
4	Helm and Kustomize configuration structure	One description of an application that serves every environment, with only the genuine differences written down separately.
5	A defined environment promotion path with approval gates	The exact same tested artifact moves dev to staging to production. Nothing is rebuilt between environments.
6	One-click rollback backed by full revision history	Recovering from a bad release is routine and fast, which is what allows the team to release more often without more risk.

1.5 The Results and What They Mean

Measure	Before	After	What that means in practice
Deployment lead time	Baseline	55% lower	A change that used to sit for hours waiting on people and manual steps now flows through automatically. Most of the saving comes from removing waiting time, not from making machines faster.
Deployment success rate	Frequent failures	Above 99%	Releases became boring. Fewer than one in a hundred needed intervention. This is mostly because every environment now uses the same tested definition and every change is validated before it is applied.
Who can deploy	Platform team only	Any developer, within guardrails	The platform team stopped being a queue. They set the rules once, and the rules are enforced automatically on every change.
Time to recover from a bad release	Manual, unpredictable	One action, minutes	Rollback is an ordinary deployment pointed at the previous revision, so it uses the same well-tested path as a normal release.
Confidence about production	Low	High	Anyone can read the Git repository and know exactly what production should look like — and the controllers guarantee that it does.

2. Background Concepts, Explained From Zero

This section explains the ideas the rest of the document depends on. If these seven ideas are clear, every later section will make sense.

2.1 What a Deployment Actually Is

A deployment is the act of taking a new version of software and making it the version that real users are talking to. It involves getting the new version onto machines, starting it, checking it is healthy, sending traffic to it, and stopping the old version. Historically a person did these steps. The whole point of this project was to make a machine do them, identically, every time.

2.2 What Kubernetes Is (In One Minute)

Kubernetes is a system that runs applications across a group of machines. You do not tell Kubernetes "start this program on server 7". You tell it "I want three copies of this application running, reachable at this address, with this much memory". Kubernetes then figures out where to put them, restarts them if they crash, and replaces them if a machine dies.

The important consequence: **Kubernetes is already built around the idea of describing a desired outcome.** You give it a description, and it works continuously to make that description true. GitOps takes that same idea and extends it one level outward: instead of a human handing Kubernetes the description, Git holds the description and a controller hands it over automatically.

- **Pod** — one running instance of your application (usually one container).
- **Deployment** — an object that says "keep N pods of this image running, and roll them out safely when the image changes".
- **Service** — a stable internal address that routes to whichever pods are currently healthy.
- **Ingress** — the rule that lets outside traffic reach a Service through a hostname and path.
- **ConfigMap / Secret** — configuration values and sensitive values handed to the application.
- **Namespace** — a folder inside the cluster used to separate teams and environments.
- **Manifest** — a YAML file describing any of the above. The cluster is, in effect, a large collection of manifests.

2.3 Declarative vs Imperative — the Central Distinction

Imperative (the old way)	Declarative (this platform)
You write the steps : "pull this image, stop the old container, start the new one, update the load balancer".	You write the end result : "version 1.4.2 should be running, with 5 replicas, reachable at pay.example.com".
If a step is skipped or run twice, the outcome differs.	Running it again changes nothing, because the end result is already true. This property is called idempotency .

Imperative (the old way)	Declarative (this platform)
You must know the current state to write the correct steps.	You do not care about the current state. The system works out the difference for you.
Recovering means reasoning about which steps to undo.	Recovering means declaring the previous end result again.
The truth lives in whoever ran the commands.	The truth lives in a file, in Git, with a full history.

Almost every benefit claimed in this project traces back to this switch. Declarative descriptions can be reviewed, tested, diffed, versioned, reverted and re-applied safely. Sequences of commands cannot.

2.4 Desired State vs Live State

Two phrases appear constantly in this document, so they are worth fixing firmly in mind.

- **Desired state** — what the system *should* look like. It lives in Git as YAML files. It is written by humans and reviewed by humans.
- **Live state** — what the system *actually* looks like right now. It lives inside the Kubernetes cluster and can be read through the Kubernetes API.

The entire platform is a machine for keeping those two things equal. When they are equal, the application is described as **Synced**. When they are not, it is **OutOfSync**, and the difference between them is called **drift**.

2.5 What GitOps Means

GitOps is the practice of using Git as the single source of truth for a system, and having automation continuously enforce it. It rests on four rules:

1. The entire desired state of the system is **declarative** — written as data, not as scripts.
2. That desired state is **versioned in Git**, so every change has an author, a time, a reason, a reviewer and an undo.
3. Approved changes are **applied automatically** — no human runs the deployment.
4. Software agents **continuously verify** the live state and correct or report any difference.

Rule 4 is the one people underestimate. A traditional pipeline deploys and then forgets. A GitOps controller never forgets — it keeps checking forever, which is what makes drift detection and self-healing possible.

2.6 Push vs Pull — and Why Pull Is More Secure

Push model (traditional CI/CD)	Pull model (GitOps, used here)
The build pipeline reaches into the cluster and applies changes.	A controller inside the cluster reaches out to Git and pulls changes.

Push model (traditional CI/CD)	Pull model (GitOps, used here)
The pipeline must hold powerful cluster credentials. Anyone who compromises the pipeline controls production.	No external system holds cluster credentials. The cluster only needs read access to a Git repository.
The cluster must be reachable from the CI system, so its API is exposed more widely.	The cluster makes only outbound connections. Its API can stay private.
If the pipeline stops running, drift goes unnoticed indefinitely.	The controller keeps checking regardless of whether anyone is building anything.
Deployment history lives in pipeline logs, which are eventually deleted.	Deployment history is the Git history, which is permanent.

2.7 Why "Self-Service" Matters

A platform is only useful if people can use it without help. "Self-service" means a developer can perform the whole journey — create a service, configure it, deploy it, promote it, roll it back — through a portal, a CLI command or a pull request, without a platform engineer being involved.

This is not the same as "no rules". The rules still exist, but they are **encoded in the platform instead of enforced by a person**. Templates produce correct configuration by default; policy checks reject unsafe configuration; access control decides who may touch production. The platform team writes the rules once, and the rules apply on every change forever after. That is what removed the queue and, with it, most of the lead time.

3. Every Component, Explained End to End

Each component below is described the same way: **what it is**, **why it is needed**, **what was configured**, **what it does in the end-to-end flow**, and **a plain analogy**. Read them in order — each one hands over to the next.

3.1 The Self-Service Platform Layer

What it is

The front door of the whole system. It is the part a developer actually touches: a web portal, a command-line tool, or both. It turns a short human request such as "create a new service called payments" or "deploy version 1.4.2 to staging" into the correct files, in the correct repository, in the correct format.

Why it is needed

GitOps on its own does not remove work — it moves the work into writing YAML. If every developer must hand-write Kubernetes manifests, Helm charts and pipeline definitions, the platform team simply becomes a YAML help desk instead of a deployment help desk. The self-service layer is what makes GitOps actually feel fast to the people using it.

What was configured

- **Golden-path templates (scaffolding).** A pre-approved starter kit for a new service: a Helm chart, Kustomize overlays for each environment, a CI pipeline definition, health-check probes, resource limits, default alerts, a dashboard, and an ownership file. A new service starts life already correct.
- **A service catalogue.** A list of every service, who owns it, which environments it runs in, and where its repository and dashboards are. This is what makes ownership answerable at 3am.
- **Guardrails.** The templates do not merely suggest good settings; the policy engine rejects configurations that lack them. Missing resource limits, missing liveness probes, images pulled from untrusted registries, and privileged containers are all refused before merge.
- **Role-based access control (RBAC).** Who may deploy to which environment. Typically: any developer may merge to development, the owning team may approve staging, and a named group must approve production.
- **Self-service actions.** Common operations exposed as buttons or commands: view live status, view the diff between Git and the cluster, trigger a sync, and roll back.

Its role end to end

The developer states an intention. The platform layer converts that intention into a **pull request** against the configuration repository. From that moment on, the change is an ordinary reviewable code change, and every downstream component treats it identically to any other change. The platform layer never talks to the cluster directly — this is deliberate, and it is what keeps a single path to production.

Analogy

It is the self-checkout at a supermarket. The rules of the shop have not changed and nothing is unsupervised — but the customer no longer queues for a cashier to do something they can safely do themselves.

3.2 Git as the Single Source of Truth

What it is

Git is a version control system: it stores files, records every change as a commit, and never forgets. In this architecture Git is promoted from "where the code lives" to "**the definition of the system**". If a setting is not in Git, that setting is not real, and the platform will actively remove it.

Why it is needed

Every property the organisation wanted from its deployment process already exists in Git for free:

Requirement	How Git provides it without extra tooling
A record of what should be running	The current contents of the main branch.
Change history and audit	The commit log: who changed what, when, and why.
Peer review before change	Pull requests, with required approvals and CODEOWNERS rules.
Undo	git revert produces a new commit restoring the previous content.
Comparing two versions	git diff — the same tool used for reviewing application code.
Access control	Branch protection rules and repository permissions.
Disaster recovery	A cluster can be rebuilt from empty by pointing a controller at the repository.

What was configured — the repository layout

Three kinds of repository, deliberately separated:

- **Application source repositories** — the actual program code. One per service. Changing this triggers a build.
- **Configuration repositories** — the desired state of each application in each environment: Helm values, Kustomize overlays, image tags, replica counts, hostnames. Changing this triggers a deployment.
- **Infrastructure repository** — the desired state of the cluster itself: namespaces, ingress controllers, DNS, certificate management, monitoring stack, quotas, cluster RBAC, operators. This is the "application and infrastructure state" phrase from the summary; both are governed by exactly the same mechanism.

A representative configuration repository looks like this:

```
config-repo/
  apps/
    payments/
      base/                <- the parts common to every environment
        deployment.yaml
        service.yaml
```

```

    kustomization.yaml
overlays/
  dev/
    kustomization.yaml    <- 1 replica, debug logging, dev hostname
    patch-resources.yaml
  staging/
    kustomization.yaml    <- 2 replicas, staging hostname
  prod/
    kustomization.yaml    <- 6 replicas, HPA, prod hostname
    patch-resources.yaml
checkout/
  ...
clusters/
  dev/      <- which apps run in the dev cluster
  staging/
  prod/
CODEOWNERS  <- who must approve changes to each path

```

Why source code and configuration are kept apart

- A configuration change (scale from 3 replicas to 6) should not require rebuilding and retesting the application.
- It keeps the application repository's history readable — it is not flooded with automated image-tag commits.
- It allows different review rules: production configuration can require a release manager, while application code requires only a peer.
- It avoids a loop where the pipeline commits to the same repository that triggers the pipeline.

Analogy

Git is the master blueprint of a building, kept in one locked cabinet. If a wall is not on the blueprint, the wall is not supposed to be there. Anyone who wants to move a wall must submit a marked-up blueprint for approval first.

3.3 The CI Pipeline (Build and Test)

What it is

Continuous Integration is the automation that runs when application code changes. Crucially, in this architecture CI **stops at the boundary of Git**. It builds and proves an artifact, then proposes a change to the configuration repository. It never touches a cluster.

The stages, in order

1. **Checkout** — fetch the exact commit that triggered the build.
2. **Build** — compile the code and produce a container image.
3. **Unit and integration tests** — fail immediately if the code is broken, so nothing further is wasted.
4. **Static analysis and dependency scanning** — check for known vulnerable libraries and unsafe code patterns.
5. **Container image scan** — check the finished image for vulnerable operating-system packages.

6. **Sign the image** — cryptographically sign it so the cluster can later verify it came from the official pipeline.
7. **Push to the container registry** — using an **immutable** tag that includes the commit hash, for example `payments:1.4.2-a91c3f`. Never `"latest"`.
8. **Open a pull request against the configuration repository** — a bot changes exactly one line: the image tag for the development environment.

Why immutable tags matter so much

If a tag such as `"latest"` or `"v1.4"` can be overwritten, then the same words in Git can mean different software on different days. Every guarantee in this document collapses at that point: the diff is meaningless, promotion proves nothing, and rollback is not reliable. Tagging every build with its commit hash means the description in Git identifies exactly one set of bytes, permanently.

Analogy

CI is the factory and the quality inspector. It produces a sealed, serial-numbered box and puts it in the warehouse. It does not decide what gets shipped to customers — it only writes a note proposing that this box be shipped.

3.4 The Container Registry

A container image is a sealed package containing the application and everything it needs to run. The registry is the storage system that holds those images. It sits between CI (which pushes images) and the clusters (which pull them).

- **Immutable tags** are enforced so a published tag can never be overwritten.
- **Digests** (a content hash such as `sha256:9f2c...`) are recorded, so a deployment can pin the exact bytes rather than trusting a name.
- **Signature verification** at admission means the cluster refuses images that the pipeline did not sign.
- **Retention rules** keep older images available long enough that rolling back several versions is always possible.

This is the reason the promotion step is trustworthy: the identical image that was tested in staging is the image that runs in production. Nothing is rebuilt in between, so nothing can differ.

3.5 Helm

What it is

Helm is a templating and packaging system for Kubernetes. Instead of writing one complete set of YAML files per environment, you write one **chart** — a set of templates with blanks in them — and one small **values file** per environment that fills in the blanks.

Its parts

Part	Plain meaning
Chart	A folder containing the templates and defaults for one application. The reusable package.

Part	Plain meaning
Chart.yaml	The chart's name, its version, and the version of the application it deploys.
templates/	The YAML files with placeholders, e.g. replicas: {{ .Values.replicaCount }}.
values.yaml	The default answers for those placeholders.
values-dev / -staging / -prod	Per-environment overrides. Only the genuine differences appear here.
Release	One installation of a chart into one namespace, with a name and a revision number.
Rendering	Combining the templates with the values to produce plain Kubernetes YAML.

```
# templates/deployment.yaml (written once)
spec:
  replicas: {{ .Values.replicaCount }}
  containers:
    - image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"

# values-dev.yaml          # values-prod.yaml
replicaCount: 1            replicaCount: 6
image:                    image:
  tag: 1.4.2-a91c3f        tag: 1.4.1-77bd21
```

Why it is needed

Without templating, three environments means three near-identical copies of every file. They inevitably diverge, because a fix applied to one copy is forgotten in the others. That divergence is one of the biggest historical causes of failed deployments, and removing it is a direct contributor to the success-rate improvement.

How it is used here

The controllers render the chart themselves — the platform does not run "helm install" from a laptop or a pipeline. In Flux this is expressed as a **HelmRelease** object; in Argo CD the Application points at a chart path and a values file. Either way, the rendered output is compared with the live cluster before anything is applied.

Analogy

A form letter. One template, and a small sheet of details per recipient. Correcting a typo in the template corrects every letter at once.

3.6 Kustomize

What it is

Kustomize solves the same problem as Helm by a different route: no templating language at all. You keep plain, valid Kubernetes YAML in a **base** folder, and each environment has an **overlay** folder describing only the patches that environment applies.

```
# base/kustomization.yaml
resources:
- deployment.yaml
- service.yaml

# overlays/prod/kustomization.yaml
namespace: payments-prod
resources:
- ../../base
replicas:
- name: payments
  count: 6
images:
- name: payments
  newTag: 1.4.1-77bd21
patches:
- path: patch-resources.yaml      # bigger CPU and memory in prod only
```

Why both Helm and Kustomize are used

Situation	Tool chosen	Reason
Third-party software (ingress controller, monitoring stack, database operator)	Helm	Vendors ship Helm charts. Using them as-is means upgrades are a version bump instead of a rewrite.
Our own services, where teams need to read and understand the YAML	Kustomize	The base files are plain Kubernetes YAML. There is no template language to learn or debug.
A vendor chart that needs a small change the chart does not expose	Both	Render the Helm chart, then apply a Kustomize patch on top of the output. This avoids forking the chart.
Environment differences that are only values (replicas, tags, hostnames)	Either	Both handle this cleanly; the team standard decides.

Supporting both was a deliberate architectural decision. Forcing one tool everywhere would have meant either rewriting every vendor chart, or pushing complex templating onto teams who did not want it.

Analogy

Helm is a form letter with blanks. Kustomize is a finished letter plus a short list of sticky notes saying "for this recipient, change these two lines".

3.7 Argo CD

What it is

Argo CD is a GitOps controller that runs inside the Kubernetes cluster. It watches Git repositories, renders the configuration, compares it to what is actually running, shows the difference visually, and applies it. It also provides the user-facing surface: a web interface, a health view, a revision history and a rollback button.

Its concepts

Concept	Plain meaning
Application	The core object. It says: this Git repository, this path, this branch or tag, deployed into this cluster and namespace. One per service per environment.
ApplicationSet	A generator that produces many Applications from one definition — for example, "create this Application for every folder under /apps, in all three environments". This is what allowed the platform to scale to many services without a person hand-writing Applications.
App-of-Apps	One parent Application whose contents are other Applications. Bootstrapping a whole cluster becomes: point Argo CD at one repository path and let it discover everything else.
Project	A boundary that restricts which repositories, clusters, namespaces and resource kinds a group of Applications may use. This is a major guardrail.
Sync	The act of applying the desired state to the cluster.
Sync policy	Automatic or manual. Development is automatic; production requires an approved merge and, optionally, a manual sync.
Sync waves	Ordering. Wave 0 might create the namespace and secrets, wave 1 the database migration job, wave 2 the application. This prevents the classic failure where an application starts before what it depends on exists.
Hooks	Jobs that run before, during or after a sync — most commonly a database migration (PreSync) or a smoke test (PostSync). A failed hook stops the rollout.
Health status	Argo CD understands what "healthy" means for each Kubernetes resource type, so a sync is not reported successful merely because the YAML was accepted.
Sync status	Synced or OutOfSync — the direct output of drift detection.
Self-heal	If enabled, any drift is automatically corrected on the next pass.
Prune	If a resource is deleted from Git, delete it from the cluster too. Without this, Git stops being a complete description.
History and rollback	Argo CD stores previous synced revisions and can redeploy any of them from the interface.

What it contributes to the results

- The **visual diff** lets a reviewer see precisely what will change in the cluster before approving — this alone prevented a large share of failed releases.

- **Health checks** turn "the deployment finished" into "the deployment is actually working", which is what makes the success-rate metric meaningful.
- **Sync waves and hooks** removed ordering-related failures, historically one of the most common causes of a broken release.
- The **rollback button** is the literal "one-click rollback" from the summary.

Analogy

A building inspector with the blueprint in hand who walks the site continuously, marks every discrepancy on a visible board, and either fixes it immediately or raises an alarm.

3.8 Flux

What it is

Flux is the other GitOps controller. It does the same fundamental job but is designed as a set of small, focused controllers with no user interface, driven entirely by Kubernetes objects. That makes it excellent for automation and for infrastructure that should run without anyone watching.

Its controllers, one by one

Controller	What it does, in plain terms
source-controller	Fetches and caches sources: Git repositories, Helm repositories and OCI artifacts. It verifies checksums and signatures, and tells the other controllers when something new has arrived.
kustomize-controller	Takes a source, runs the Kustomize build, compares the result with the cluster, and applies the difference. It also handles pruning and health checks.
helm-controller	Manages HelmRelease objects: installs, upgrades, tests and — importantly — automatically rolls back a Helm release that fails to become healthy.
image-reflector-controller	Watches the container registry and keeps a list of the available image tags.
image-automation-controller	When a newer tag matching a policy appears, it writes that tag into the configuration repository and commits it. This is automation that closes the loop back into Git rather than around it.
notification-controller	Sends events outward (Slack, Teams, webhooks) and receives inbound Git webhooks so a merge can trigger reconciliation instantly instead of waiting for the next poll.

Its objects

- **GitRepository / HelmRepository / OCIRepository** — where to fetch from, which branch or tag, how often, and how to verify it.

- **Kustomization** — which path to build, into which namespace, at what interval, whether to prune, and which other Kustomization it depends on.
- **HelmRelease** — which chart, which version, which values, and what to do if the upgrade fails (retry, then roll back).
- **ImagePolicy / ImageUpdateAutomation** — which tags are acceptable and where to write them back in Git.

Analogy

A team of specialist maintenance robots. Each has one narrow job, none of them needs a supervisor, and together they keep the building matching the blueprint around the clock.

3.9 Why Both Argo CD and Flux Were Used

Running two controllers is a deliberate choice, not duplication. They are given clearly separated responsibilities so they never manage the same resource.

Dimension	Argo CD	Flux
Primary users	Application developers and release managers	Platform and infrastructure engineers
Typical scope	Business applications across dev, staging and production	Cluster add-ons, operators, bootstrap and shared infrastructure
Interface	Rich web UI with visual diff, health tree and rollback button	Kubernetes objects and CLI; no UI by design
Biggest strength here	Human visibility, approval workflow, one-click rollback, ApplicationSet scaling	Unattended automation, image tag automation writing back to Git, lightweight multi-cluster bootstrap
Ownership boundary	Owns everything under /apps	Owns everything under /clusters and /infrastructure

The rule that keeps two controllers safe

Exactly one controller owns any given resource. Ownership is decided by repository path and enforced by Argo CD Projects and Flux Kustomization scopes.

If two controllers managed the same object with different desired states, each would repeatedly undo the other — an endless fight that would look like constant, unexplained drift. Preventing that is an architecture decision, not a configuration detail.

3.10 Automated Reconciliation

What it is

Reconciliation is the repeating loop that keeps the live state equal to the desired state. It is the engine of the entire platform, and it runs continuously whether or not anyone is deploying anything.

The loop, step by step

1. **Fetch.** Pull the latest commit of the tracked branch or tag from Git. Verify the commit signature if signing is required.
2. **Render.** Run Helm and/or Kustomize to turn templates into final, plain Kubernetes manifests.
3. **Read live state.** Ask the Kubernetes API what those objects currently look like.
4. **Diff.** Compare desired against live, field by field. Fields the cluster legitimately manages itself (such as a status block, or a replica count owned by an autoscaler) are ignored so they do not register as false drift.
5. **Decide.** No difference: record Synced and stop. Difference found: continue if the sync policy allows it, otherwise mark OutOfSync and notify.
6. **Apply.** Apply only the objects that changed, respecting sync-wave order, and prune objects that were removed from Git.
7. **Verify.** Wait for the affected resources to report healthy — pods ready, rollout complete, checks passing.
8. **Report.** Record the outcome, emit notifications, and update the metrics.
9. **Wait, then repeat.** Sleep until the configured interval (typically one to five minutes) or until a Git webhook arrives.

What triggers a pass

- **A timer** — the safety net. Guarantees drift is found within one interval even if every notification mechanism fails.
- **A Git webhook** — the speed path. A merge notifies the controller immediately, so the change usually starts applying within seconds.
- **A manual sync** — a human presses Sync, typically for a gated production release.
- **A watch event** — a resource the controller manages changed in the cluster, prompting an immediate re-check.

Why this is the biggest single contributor to the results

The old lead time was dominated by **waiting** — waiting for a person to be free, waiting for a change window, waiting for someone to remember. Reconciliation removes waiting entirely, because the trigger is a merge and the actor is a machine that is always available. It also removes the whole class of failures caused by a deployment step being skipped, run twice, or run in the wrong order.

3.11 Drift Detection

What drift is

Drift is any difference between what Git says should be running and what is actually running. It is the natural, inevitable decay of a system that people can touch directly.

Where drift comes from

Cause	Typical example	How the platform responds
Manual emergency fix	An engineer scales a deployment by hand at 2am to survive a traffic spike.	Detected on the next pass. Either reverted automatically, or flagged so the fix can be written back into Git properly.
Well-meaning experiment	Someone edits an environment variable to test a theory and forgets to undo it.	Reverted automatically. The cluster stops accumulating undocumented changes.
Another controller	A horizontal pod autoscaler changes the replica count.	Not treated as drift — that field is deliberately excluded from comparison so the two systems do not fight.
Something was deleted	A resource is removed by accident or by a failed script.	Recreated automatically from Git on the next pass.
A default was applied	Kubernetes or a mutating webhook adds fields the author never wrote.	Ignored via configured diff-normalisation rules, to avoid permanent false OutOfSync noise.

Auto-heal versus alert-only

Both behaviours are useful, and the choice is per-environment:

- **Development and staging: auto-heal on.** Correctness matters more than preserving someone's experiment. Manual changes vanish quickly, which trains everyone to make changes in Git.
- **Production: usually alert-first.** During a live incident, an engineer may legitimately need a temporary manual change. Automatically undoing it mid-incident would make things worse. So production raises an OutOfSync alert loudly, and the expectation is that the fix is committed to Git promptly. Auto-heal can be re-enabled once the incident closes.

Why drift detection is worth so much

Undetected drift is what made the old environments untrustworthy. Once staging and production quietly differ, testing in staging proves nothing, and deployments that "worked in staging" fail in production. Continuously detecting and eliminating drift is what made the environments genuinely comparable — and comparable environments are the reason the deployment success rate could exceed 99%.

Analogy

A smoke alarm that also puts the fire out. It is always on, it does not need to be told to check, and it notices the problem long before a person walks past the room.

3.12 Environment Promotion

What it is

Promotion is moving one specific, already-built, already-tested version of software forward through the environments: development, then staging, then production.

The essential rule

Build once, promote the same artifact

The image is built exactly once, by CI, and given an immutable tag. That identical image is what runs in development, in staging and in production.

Nothing is ever rebuilt for a specific environment. If it were, the thing tested would not be the thing shipped, and every test result would be an approximation.

Therefore **a promotion is not a build — it is a one-line change in Git** that points the next environment at an image that already exists.

What actually happens at each step

1. **Into development.** CI opens an automated pull request changing the dev image tag. Checks pass, it merges automatically, and the controller syncs it. No human is involved.
2. **Development verification.** Smoke tests and health checks run. If the application does not become healthy, promotion stops here and the pull request for staging is never opened.
3. **Into staging.** A pull request copies the same tag into the staging values file. It requires review by the owning team. On merge, the controller syncs staging.
4. **Staging verification.** Integration tests, contract tests and performance checks run against a cluster shaped like production.
5. **Into production.** A pull request copies the same tag into the production values file. Branch protection requires an approval from the release-manager group and CODEOWNERS sign-off. Policy checks re-run against the production configuration.
6. **Production rollout.** On merge the controller syncs. In production the rollout is usually progressive: a canary receives a small percentage of traffic while metrics are watched, and only then does it scale to the full fleet.

Why promotion through Git is better than promotion through a pipeline

- The record of what is in each environment is a **file**, not a pipeline run that will be archived and deleted.
- Approval is a pull-request approval — a mechanism the organisation already trusts, audits and understands.
- The diff shown to the approver is the literal change: one image tag. It is very hard to approve something you did not intend.
- Rollback is symmetric: reverting the promotion commit is exactly as easy as making it.
- Comparing two environments is a Git diff between two folders, which anyone can run.

3.13 One-Click Rollback

What it is

The ability to return to the previously working version with a single action, in minutes, without improvisation.

The two routes, and when each is used

Route	How it works	When to use it
Git revert (preferred)	Revert the commit that introduced the change. This creates a new commit restoring the previous content. The controller sees the new desired state and syncs the cluster back.	Almost always. Git stays truthful, the revert is auditable, and self-heal will not undo it.
Controller history rollback (fastest)	Press Rollback in Argo CD, or let the Flux helm-controller roll a failed HelmRelease back automatically. The controller re-applies a stored previous revision immediately.	During an incident when seconds matter. Git must then be reverted too, otherwise the controller will re-apply the broken version on the next pass.

Why it is fast and safe

- The previous revision is stored in full, so there is nothing to reconstruct under pressure.
- Rollback travels the **same code path as a normal deployment**. There is no separate emergency script that is rarely exercised and therefore untrustworthy.
- The old container image is still in the registry, because retention rules guarantee it.
- Auto-sync is paused during a manual history rollback and re-enabled afterwards, so the controller does not immediately reapply the broken version while the operator is working.
- Health checks confirm the rollback actually restored a healthy state, rather than assuming it.

The honest limitation

Rollback restores the **application** state, not the **data** state. A schema migration that dropped a column is not undone by reverting a Git commit.

This is handled with the expand-and-contract pattern: migrations are written so that the old and new application versions can both run against the same schema for a period. Destructive changes are made only after the new version is proven. This is a discipline the platform supports, not a problem it solves by itself.

Why fast rollback increases speed overall

This is the counter-intuitive part. When rolling back is slow and frightening, teams protect themselves by releasing rarely and in large batches — which makes each release riskier and harder to diagnose, which makes them release even less often. Making rollback trivial breaks that cycle. Small, frequent, low-stakes releases are exactly what produces both the shorter lead time and the higher success rate.

3.14 Secrets Management

GitOps creates an obvious question: if everything is in Git, what about passwords and API keys? Plain secrets must never be committed. Two standard solutions were used.

Approach	How it works	Trade-off
Sealed Secrets	The secret is encrypted with a public key before being committed. Only a controller inside the target cluster holds the private key and can decrypt it. The encrypted blob in Git is useless to anyone else.	Keeps Git genuinely complete, so a cluster can be rebuilt from the repository alone. Requires care around key rotation and backup.
External Secrets Operator	Git stores only a pointer — "fetch the value named payments/db-password from the vault". An operator in the cluster fetches the real value at runtime and creates the Kubernetes Secret.	The secret never enters Git in any form, and rotation happens in the vault without a commit. Adds a runtime dependency on the vault.

Either way, the reviewable, versioned workflow is preserved for everything else, and no human needs standing access to production secrets in order to deploy.

3.15 Policy, Validation and Guardrails

Guardrails are the reason self-service does not become a hazard. They are applied at three separate points, deliberately, because each catches what the others cannot.

Where	What runs	What it catches
In the pull request (cheapest)	YAML linting, Kubernetes schema validation (kubecconform), Helm and Kustomize dry-run rendering, policy unit tests, and a preview of the cluster diff.	Typos, invalid fields, broken templates, missing resource limits, missing probes, and any unintended change to another service.
At sync time	The controller performs a server-side dry run before applying, and enforces Project and Kustomization boundaries.	A change that is individually valid but attempts to write outside the boundary it is allowed to touch.
At admission (last line)	OPA Gatekeeper or Kyverno inside the cluster.	Anything that reaches the API server by any route at all — including a manual kubectl command — that violates organisational rules.

Typical enforced rules: every container declares CPU and memory limits; every workload has liveness and readiness probes; images come only from the approved registry and carry a valid signature; no privileged containers; required ownership and cost-centre labels are present; production namespaces require a minimum replica count and a pod disruption budget.

The design principle is **shift left**: catch a problem in the pull request where it costs a minute to fix, rather than in production where it costs an incident. This is a large, direct contributor to the above-99% success rate.

3.16 Observability, Notifications and Metrics

A platform that cannot be measured cannot be improved, and the headline numbers in the summary had to come from somewhere real.

- **Application health** — Prometheus and Grafana collect metrics; alerts fire on error rate, latency and saturation. Argo CD and Flux both expose their own metrics, so the health of the delivery system itself is monitored too.
- **Sync and drift events** — every reconciliation outcome is emitted as an event: synced, failed, out of sync, healed, rolled back.
- **Notifications** — Slack messages on production sync, on any sync failure, and on any production drift. Notifications go to the owning team, taken from the ownership metadata created by the golden-path template.
- **DORA metrics** — the four standard delivery measures are derived automatically: deployment frequency, lead time for change, change failure rate, and time to restore service. Lead time is measured as the elapsed time from first commit to healthy in production; success rate is measured as the proportion of production syncs that reached a healthy state without a rollback. Because both are computed from Git timestamps and controller events, they are objective rather than self-reported.

3.17 Access Control and the Security Model

- **No external system holds cluster credentials.** The controllers run inside the cluster and pull. The CI pipeline can only write to Git. If CI is compromised, the attacker can propose a change — but it still has to pass review, policy checks and approval.
- **Access is expressed in two layers.** Git branch protection and CODEOWNERS decide who may change which environment. Argo CD Projects and Kubernetes RBAC decide what a controller or user may do inside the cluster.
- **Single sign-on** governs who can view, sync or roll back which application, so an engineer on one team cannot sync another team's production service.
- **Every change to production is attributable** to a person, a commit, an approver and a timestamp — which turns an audit from an investigation into a query.
- **Optional commit signing** means the controller will only apply commits signed by a trusted key, closing the gap where someone pushes directly to a branch.

4. The End-to-End Workflow, Stage by Stage

This section follows one real change all the way from a developer's keyboard to customers using it, and then onward to reconciliation and rollback. Nothing is skipped. The running example is a change to a service called **payments**, released as image tag **payments:1.4.2-a91c3f**.

4.1 The Whole Workflow at a Glance

#	Stage	Who or what acts	Typical time
0	Onboarding a new service (once per service)	Developer via the self-service portal	Minutes
1	Write code and open a pull request	Developer	Varies
2	CI builds, tests, scans, signs and pushes the image	CI pipeline	5-15 min
3	Automated pull request updates the dev image tag	CI bot	Seconds
4	Automated checks run on the configuration change	Validation and policy tools	1-3 min
5	Merge to the main branch of the config repository	Automatic for dev; reviewed for higher environments	Seconds to hours
6	Controller is notified of the new commit	Git webhook, or the polling timer	Seconds
7	Controller renders Helm and Kustomize into final manifests	Argo CD or Flux	Seconds
8	Controller diffs desired state against live state	Argo CD or Flux	Seconds
9	Controller applies only the difference, in wave order	Argo CD or Flux	Seconds
10	Kubernetes performs the rolling update	Kubernetes	1-5 min
11	Health is verified and the result is reported	Controller, then notifications	Seconds
12	Promotion to staging, then to production, with approval	Team, then release manager	Minutes to hours
13	Continuous reconciliation and drift correction	Controller, forever	Every 1-5 min
14	Rollback, if it is ever needed	Any authorised engineer, one action	1-3 min

4.2 Stage 0 — Onboarding a New Service (Once)

- **Trigger:** a developer needs a new service to exist.
- **Action:** they open the developer portal, or run a single CLI command, and answer a few questions — service name, owning team, language, which environments it needs, expected traffic.

- **What the platform generates:** an application repository seeded with a working skeleton and a CI pipeline; a folder in the configuration repository containing a base and one overlay per environment; an Argo CD Application (or an ApplicationSet entry that produces one); default alerts and a dashboard; an entry in the service catalogue recording ownership.
- **What the developer never has to do:** learn Kubernetes YAML, request cluster access, ask a platform engineer for anything, or copy configuration from another team and hope it was correct.
- **Output:** a pull request containing all of the above, ready for review.
- **Why this stage matters:** every service starts correct, with limits, probes, labels, alerts and ownership already in place. Most historical production incidents traced back to a service that was set up in a hurry and never revisited.

4.3 Stage 1 — The Developer Writes Code

- The developer works only in the **application repository**. They do not open the configuration repository, and they do not think about clusters.
- They commit, push a branch, and open a pull request. A peer reviews the code as normal.
- On merge to the main branch, the CI pipeline starts automatically.

4.4 Stage 2 — CI Builds and Proves the Artifact

1. Checks out the exact merged commit, a91c3f.
2. Compiles the code and runs unit tests. A failure here stops everything, and nothing further is attempted.
3. Runs integration tests against ephemeral dependencies.
4. Runs dependency and static analysis scans for known vulnerabilities.
5. Builds a container image.
6. Scans the finished image for vulnerable operating-system packages. A critical finding fails the build.
7. Signs the image so the cluster can verify its origin later.
8. Pushes it to the registry as payments:1.4.2-a91c3f — an immutable tag containing the commit hash, and never "latest".

The hard boundary

This is where CI stops. The pipeline has **no credentials for any cluster** and cannot deploy anything.

Its only remaining power is to propose a change to Git — which is subject to exactly the same review as a human proposal.

4.5 Stage 3 — The Automated Configuration Pull Request

- A bot opens a pull request against the **configuration repository**.
- The change is one line: in apps/payments/overlays/dev, the image tag becomes 1.4.2-a91c3f.

- The pull request body links back to the commit, the build, the test results and the scan report, so a reviewer can see the provenance without leaving the page.
- (Alternatively, the Flux image-automation-controller performs this write itself when it detects a new tag matching the configured policy. The outcome is identical: the new desired state is proposed as a commit.)

4.6 Stage 4 — Automated Checks on the Configuration Change

Every check below runs before a human is asked to look at anything. Each one exists because it catches a class of failure that used to reach production.

Check	What it proves
YAML lint	The file is syntactically valid and consistently formatted.
Schema validation (kubecoreform)	Every field is real and correctly typed for the Kubernetes version in use. Catches misspelled fields that Kubernetes would silently ignore.
Helm / Kustomize dry-run render	The templates still build. Catches a broken template before it ever reaches a cluster.
Policy tests (OPA / Kyverno)	The result satisfies organisational rules: limits, probes, approved registry, required labels, no privileged containers.
Diff preview	A comment on the pull request showing exactly which cluster objects would change. This is the single most valuable check — a reviewer approves a visible outcome, not an intention.
Image provenance	The referenced tag exists in the approved registry and carries a valid signature.
Blast-radius check	The change touches only the paths it claims to. Catches an accidental edit to another team's service.

4.7 Stage 5 — Review and Merge

- **Development:** if all checks pass, the pull request merges automatically. No human is required, because the checks have already proven more than a human reviewer would notice in a one-line tag change.
- **Staging:** requires approval from the owning team, enforced by CODEOWNERS on the /staging path.
- **Production:** requires approval from the release-manager group, plus a passing set of production-specific policy checks. Branch protection makes this unbypassable — including for administrators.
- **The moment of merge is the moment of release.** There is no separate "deploy" action anywhere in this workflow. This is the conceptual heart of GitOps.

4.8 Stage 6 — The Controller Learns About the Commit

- **Fast path:** Git sends a webhook to the controller the instant the merge lands. Reconciliation begins within a second or two.
- **Safety net:** independently, the controller polls the repository on a fixed interval (commonly one to five minutes). If the webhook is lost, dropped or misconfigured, the change is still picked up on the next poll.
- **Why both exist:** the webhook provides speed; the timer provides a guarantee. A system that relies only on webhooks silently stops delivering when a webhook fails, and nobody notices until something is badly wrong.

4.9 Stage 7 — Rendering the Manifests

1. The controller fetches the repository at the exact commit and verifies its integrity.
2. It reads the Application (Argo CD) or Kustomization / HelmRelease (Flux) definition to learn which path to build and where to deploy it.
3. It runs Helm template rendering and/or the Kustomize build, combining the base with the correct environment overlay.
4. Out comes a set of ordinary Kubernetes manifests — Deployment, Service, Ingress, ConfigMap, HorizontalPodAutoscaler and so on — with every placeholder filled in.
5. This rendering happens **inside the cluster, from Git**. It is not something a person ran once on a laptop, which means it is repeatable and identical every time.

4.10 Stage 8 — The Diff

- The controller reads the current live version of each rendered object from the Kubernetes API.
- It compares desired against live, field by field.
- Fields legitimately owned by the cluster are normalised away so they never appear as drift: status blocks, cluster-assigned IP addresses, default values injected by the API server, mutations added by webhooks, and replica counts owned by an autoscaler.
- The result is a precise list: objects to create, objects to update (and which fields), and objects to delete because they were removed from Git.
- **If the list is empty, nothing happens at all.** This is what makes the loop safe to run every minute forever.

4.11 Stage 9 — Applying the Change

1. **Sync waves run in order.** Wave 0 might create the namespace, secrets and configuration. Wave 1 runs the database migration job. Wave 2 updates the Deployment. Wave 3 updates the Ingress. Each wave must succeed before the next begins.
2. **PreSync hooks run first.** Typically the schema migration. If it fails, the sync stops here and the application is never updated — the old version keeps running against the old schema, which is the correct and safe outcome.
3. **Only the differing objects are applied.** Untouched objects are left completely alone, which keeps the blast radius of any change as small as the change itself.

4. **Server-side apply** is used so that field ownership is tracked and the controller does not clobber fields legitimately managed by something else.
5. **Pruning** removes objects that no longer appear in Git — with protective settings so that a mistaken deletion cannot silently wipe out a namespace.

4.12 Stage 10 — Kubernetes Performs the Rolling Update

- Kubernetes creates pods running the new image alongside the existing ones.
- A new pod receives no traffic until its **readiness probe** passes, so users never reach a half-started instance.
- Once a new pod is ready, one old pod is removed. This repeats until every pod runs the new version.
- If new pods never become ready, the rollout **stalls rather than proceeding**. The old version is still serving traffic, so users are unaffected while the problem is investigated.
- In production this is often wrapped in a progressive rollout: the canary takes a small share of traffic first, error rate and latency are watched automatically, and the rollout only continues if the metrics stay healthy.

4.13 Stage 11 — Verification and Reporting

- The controller waits for its health assessment to turn green — a Deployment is healthy only when the rollout is complete and the required replicas are available.
- PostSync hooks run: smoke tests against the newly deployed version.
- The Application is marked **Synced and Healthy**, and the revision is recorded in the history so it can be rolled back to later.
- A notification goes to the owning team's Slack channel.
- Metrics are updated: lead time for this change and the success or failure of this sync feed directly into the reported numbers.
- If health is not reached within the configured timeout, the sync is marked **Degraded**, an alert fires, and — for a Flux HelmRelease — an automatic rollback to the last healthy release is triggered.

4.14 Stage 12 — Promotion Onward

Stages 3 through 11 now repeat for staging, and then again for production. The only differences are which folder is edited, who must approve, and how the rollout is performed.

	Development	Staging	Production
Who approves	Nobody — automatic	Owning team	Release-manager group
Sync policy	Automatic	Automatic on merge	Automatic on merge, or manual sync
Self-heal	On	On	Alert-first during incidents

	Development	Staging	Production
Rollout style	Straight rolling update	Rolling update	Canary, then progressive
Tests run after	Smoke tests	Integration, contract, performance	Smoke tests and live metric analysis
Image	payments:1.4.2-a91c3f	the identical image	the identical image

4.15 Stage 13 — Continuous Reconciliation Forever

The change is live, but the workflow does not end. From this point on, and for as long as the application exists, the controller re-runs the loop from Stage 7 every interval, whether or not anyone is deploying.

- If reality still matches Git, the pass takes a fraction of a second and nothing happens.
- If someone edits the cluster by hand, the difference is found within one interval.
- In development and staging, the difference is corrected automatically — the hand edit simply disappears.
- In production, an OutOfSync alert is raised so a human can decide whether to revert it or to commit it properly to Git.
- If a resource is deleted by accident, it is recreated from Git.
- If a cluster is rebuilt from scratch, pointing the controller at the repository restores everything.

4.16 Stage 14 — Rollback

1. **Detection.** An alert fires, or the canary analysis fails, or the health check never turns green.
2. **Decision.** Roll forward with a fix, or roll back. Rollback is chosen whenever the fix is not immediately obvious, because rollback is cheap.
3. **Action.** Either revert the promotion commit in Git, or press Rollback in Argo CD for immediate effect.
4. **Reconciliation.** The controller sees the previous known-good desired state and syncs the cluster to it, using the same path as any normal deployment.
5. **Recovery.** Kubernetes rolls the previous pods back in. Health checks turn green, typically within one to three minutes.
6. **Cleanup.** If the history-rollback route was used, Git must also be reverted so the two agree again; otherwise the next reconciliation pass would reapply the broken version.
7. **Record.** The event is notified, added to the audit trail, and counted in the change-failure-rate metric.

4.17 A Worked Timeline: Before and After

This is where the 55% figure comes from. Note that the machine time barely changed — what disappeared was **waiting**.

Step	Before (manual)	After (this platform)
Code merged	0 min	0 min
CI build, test, scan	12 min	12 min
Deployment request raised and queued	90 min (waiting for a person)	0 min (a bot opens the pull request)
Configuration prepared by hand	20 min	0 min (generated)
Validation and review	15 min, done by eye	2 min, automated, plus a visible diff
Applied to development	10 min of manual commands	30 seconds, automatic
Verified in development	15 min, manual checking	3 min, automated health and smoke tests
Promotion to staging	45 min, repeat the manual work	2 min, a one-line pull request
Promotion to production	60 min, change window and manual steps	10 min, approval plus a canary rollout
Total	about 4 hours 27 min	about 1 hour 59 min
Reduction		roughly 55%

4.18 What Goes Wrong at Each Stage, and What Happens Then

Stage	Failure	Automatic response	Effect on users
CI build or test	Code is broken	Pipeline stops. No image is produced, so no image can be deployed.	None
Image scan	Critical vulnerability found	Build fails and the image is never published.	None
Config validation	Invalid or non-compliant YAML	Pull request is blocked with a specific error.	None
Diff preview	The change would touch something unexpected	The reviewer sees it and rejects the pull request.	None
PreSync hook	Database migration fails	Sync halts before the application is updated. The old version keeps running against the old schema.	None
Rolling update	New pods never become ready	The rollout stalls. Old pods continue serving traffic.	None
Health check	Application starts but is unhealthy	Marked Degraded, alert fires, Flux HelmRelease rolls back automatically.	Brief, if any

Stage	Failure	Automatic response	Effect on users
Canary analysis	Error rate rises on the canary	Progressive rollout is aborted and traffic returns to the stable version.	Small share, briefly
After release	A problem appears later	Alert fires; one-click rollback restores the previous version in minutes.	Minutes
Drift	Someone changes the cluster by hand	Detected within one interval; corrected or alerted depending on the environment.	None

Read that table as a whole and the success-rate result stops being surprising. There are eight independent places where a bad change is stopped **before** it can affect a user, and each one is automatic and always on.

5. The Architecture Diagram, Explained in Full

The architecture is presented as seven figures rather than one crowded picture, so that every label stays readable. Together they cover the same single architecture from top to bottom:

- **Figure 1** — the whole system compressed into six steps.
- **Figure 2A** — the people and the self-service front door (layers 1 and 2).
- **Figure 2B** — the build pipeline and Git as the source of truth (layers 3 and 4).
- **Figure 3** — Part B of the detailed architecture: from Git to running software (layers 5 to 7).
- **Figure 4** — the reconciliation and drift-detection loop (layer 8).
- **Figure 5** — how a release is promoted between environments.
- **Figure 6** — the supporting services every layer relies on (layer 9).
- **Figure 7** — the rollback path.

5.1 How to Read These Diagrams

- **Read top to bottom.** Every diagram flows downward, in the direction a change actually travels.
- **Grey outer bands are layers.** A layer is a group of components with one shared responsibility.
- **Coloured boxes are components.** The colour identifies which layer a box belongs to, so the same component keeps the same colour across all seven figures.
- **Arrows are handovers.** The label on an arrow says what is being handed to the next layer.
- **Position means order, not network topology.** Two boxes side by side are peers in the same stage, not necessarily connected to each other.

5.2 Figure 1 — The Whole System in One Picture

This is the entire platform on one page. Read the left column top to bottom, then cross to the right column and continue. Every figure that follows is a magnification of one part of this picture.

Figure 1 - The Whole System in One Picture

A change travels in one direction: person -> Git -> controller -> cluster. Nothing skips a step.

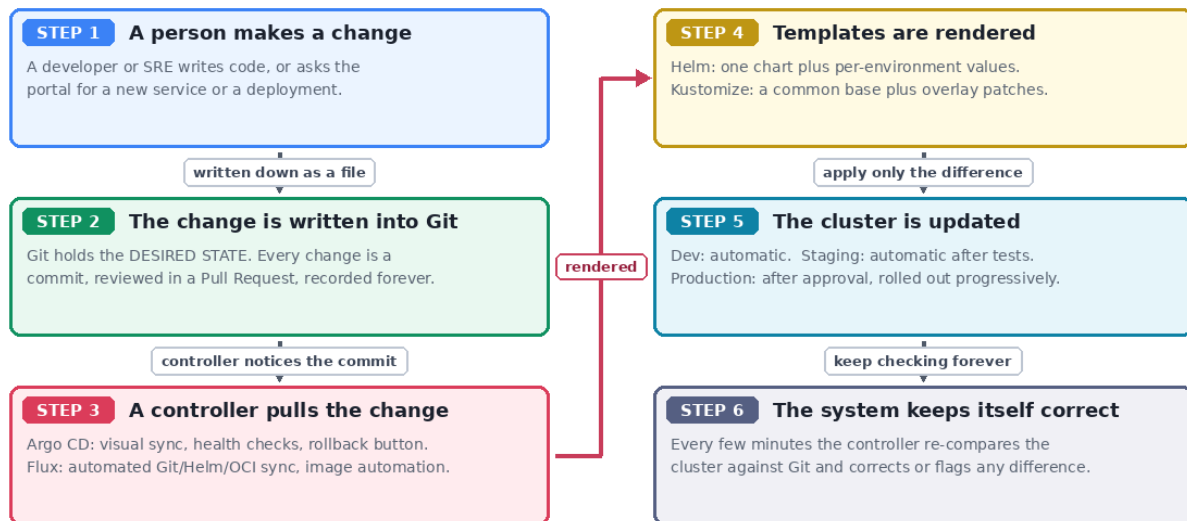


Figure 1: the entire platform reduced to six steps. Every later figure is a magnification of one part of this.

Figure 1, box by box

Element	What it represents	Why it is where it is
Step 1 — Developer or SRE	The only place a human intention enters the system.	Every change has exactly one entry point. There is no side door that bypasses the rest of the diagram.
Step 2 — Git	The written-down desired state, reviewed and permanently recorded.	It sits between the person and all automation. Nothing downstream can act until an intention has become a reviewed commit.
Step 3 — Argo CD and Flux	The controllers that read Git and act on the cluster.	They are drawn below Git and above the cluster because they only ever pull downward — Git tells them what to do, and they never take instructions from anywhere else.
Step 4 — Helm and Kustomize	The rendering step that turns templates into plain YAML.	It sits inside the controllers' work, not before them, because rendering happens in the cluster from Git — not on somebody's laptop.
Step 5 — The three clusters	The live state: what is really running.	They are at the bottom because they are the destination, and because they never push anything back upward.
Step 6 — Reconciliation and drift detection	The loop that keeps checking forever.	It is drawn last because it never finishes. It is the only element in the diagram that has no end.

Figure 1, arrow by arrow

- **"the change is written down as a file"** — the transition from human intention to reviewable data. This is the moment the change becomes auditable.
- **"the controller notices the new commit"** — a webhook or a poll. Note the direction: the cluster reaches out to Git. Git never reaches into the cluster.
- **"templates are turned into real Kubernetes YAML"** — rendering, performed by the controller.
- **"apply only the difference to the cluster"** — the word **only** matters. Untouched objects are not re-applied, which keeps the blast radius as small as the change.
- **"keep checking forever"** — the handover from a one-off deployment into a permanent, continuous guarantee.

5.3 Figures 2A and 2B — Architecture Part A: From Person to Git

Part A is shown as two figures so that every label stays readable. Together they cover layers 1 to 4 — everything that happens **before** anything touches a cluster.

Figure 2A - Architecture Layers 1 and 2

The people who use the platform, and the self-service front door they use.

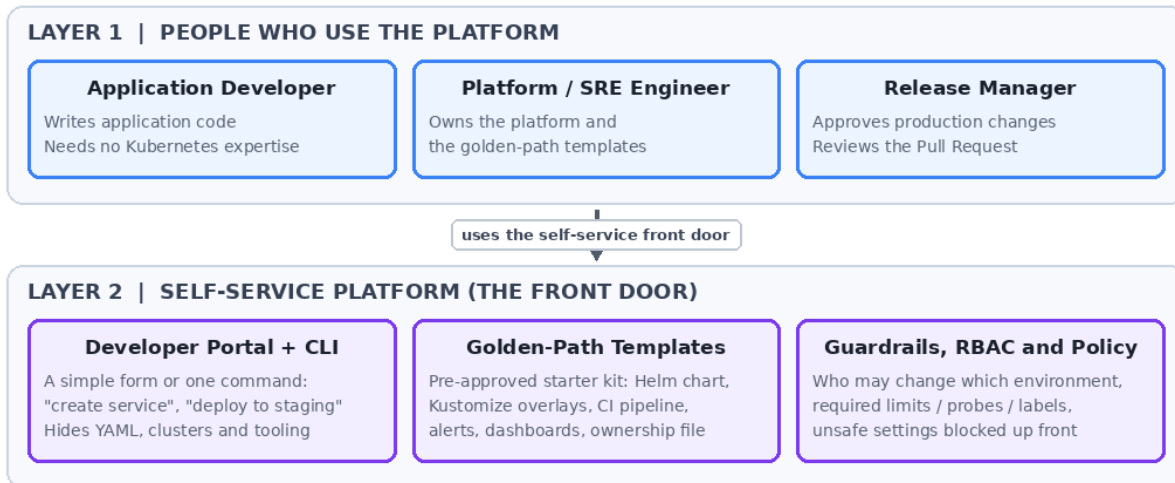


Figure 2A: layers 1 and 2 — the people, and the self-service front door they use.

Layer 1 — People who use the platform

- **Application Developer** — writes features. Deliberately requires no Kubernetes knowledge; if the platform demanded it, self-service would not be real.
- **Platform / SRE Engineer** — owns the platform itself and the golden-path templates. They set the rules once instead of executing deployments repeatedly.
- **Release Manager** — approves production promotions. Their approval is a pull-request approval, not a manual deployment.

The arrow leaving this layer ("uses the self-service front door") is important for what it excludes: none of these three people run `kubectl` against a cluster in the normal flow. That is what makes the process repeatable regardless of who is on duty.

Layer 2 — The self-service platform

- **Developer Portal + CLI** — turns a short request into the correct files. It is the usability layer, and it is the reason GitOps did not simply become "now everyone writes YAML".
- **Golden-Path Templates** — the pre-approved starter kit. A new service arrives with limits, probes, alerts, dashboards and ownership already correct.
- **Guardrails, RBAC and Policy** — the rules that make self-service safe: who may change which environment, and which configurations are refused outright.

The arrow leaving this layer ("scaffolds files and opens a Pull Request") is the key design decision of the whole platform. The portal produces a **pull request**, not a deployment. It has no cluster access at all, so there is exactly one path to production and no privileged shortcut.

Figure 2B - Architecture Layers 3 and 4

The build pipeline, and Git as the single source of truth. Nothing here touches a cluster.

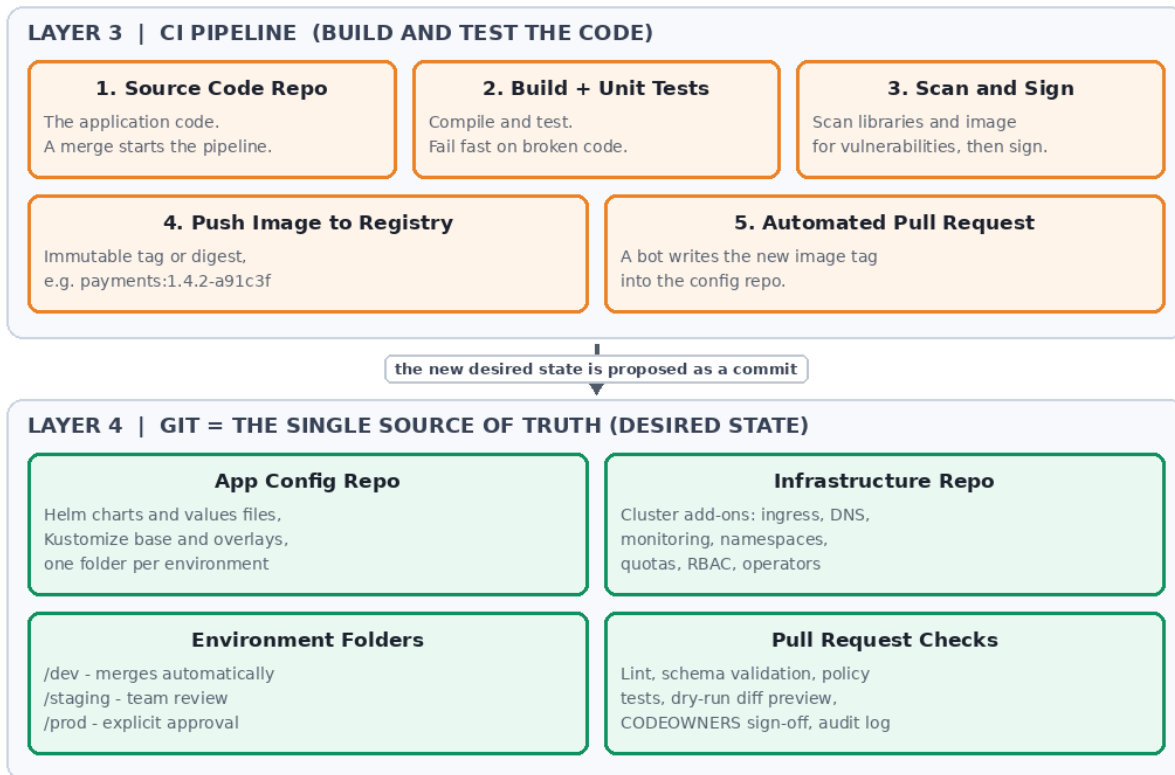


Figure 2B: layers 3 and 4 — the build pipeline, and Git as the single source of truth.

Layer 3 — The CI pipeline

The five numbered boxes run in order, and each one exists to stop a specific class of failure before it can travel further:

- 1. Source Code Repo** — the trigger. A merge here starts the pipeline.
- 2. Build + Unit Tests** — fails fast, so no effort is wasted on broken code.
- 3. Scan and Sign** — proves the artifact is not carrying known vulnerabilities, and stamps it so the cluster can verify its origin later.
- 4. Push Image to Registry** — with an immutable, commit-stamped tag. Everything downstream depends on that tag meaning exactly one thing forever.
- 5. Automated Pull Request** — the handover into Git. The pipeline writes one line and stops.

The arrow leaving this layer ("the new desired state is proposed as a commit") marks the hard security boundary. CI holds no cluster credentials. Its maximum power is to **propose**.

Layer 4 — Git, the single source of truth

- App Config Repo** — the desired state of every application in every environment.

- **Infrastructure Repo** — the desired state of the clusters themselves. Its presence in the same layer is the point: applications and infrastructure are governed by one identical mechanism, which is what the phrase "application and infrastructure state" in the summary refers to.
- **Environment Folders** — dev, staging and prod as separate paths, so different approval rules can be attached to each one.
- **Pull Request Checks** — lint, schema validation, policy tests, a rendered diff preview and CODEOWNERS sign-off. This is the busiest quality gate in the architecture and the largest single contributor to the success-rate improvement.

The arrow leaving this layer ("merged commit") continues into Figure 3. In the running system that boundary is a webhook — the merge notifies the cluster, and the cluster comes to collect the work.

5.4 Figure 3 — Architecture Part B: From Git to Running Software

Figure 3 - Architecture Part B: From Git to Running Software

Layers 5 to 7. The cluster PULLS the change; no pipeline is given cluster credentials.

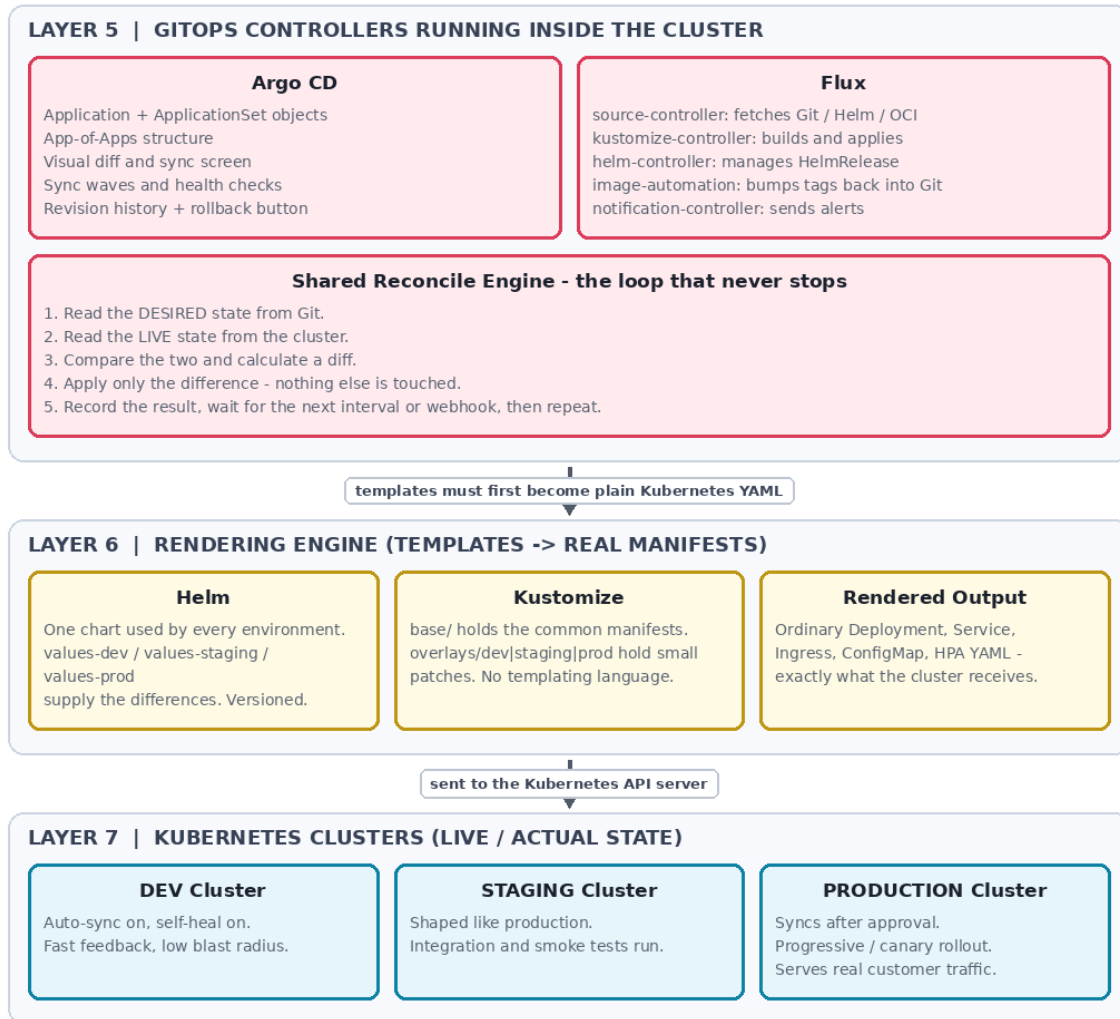


Figure 3: layers 5 to 7. The cluster pulls the change; no external system is given cluster credentials.

Layer 5 — The GitOps controllers

The two controller boxes list what each one contributes. **Argo CD** provides the human-facing surface — Application and ApplicationSet objects, the App-of-Apps structure, the visual diff, sync waves, health checks, and the revision history that makes one-click rollback possible. **Flux** provides the unattended automation — the source, kustomize, helm, image-automation and notification controllers.

The **Shared Reconcile Engine** box spans the full width on purpose. It is not a third product; it is the five-step behaviour that both controllers implement and that the entire architecture depends on: read desired, read live, diff, apply the difference, repeat. Everything above this box exists to produce a trustworthy desired state, and everything below it exists to receive the result.

The critical property of this layer is its direction. The controllers run inside the cluster and reach outward to Git. Nothing outside ever reaches in. This is why the cluster API can stay private, why no pipeline holds production credentials, and why a compromised build system still cannot deploy anything by itself.

Layer 6 — The rendering engine

- **Helm** — one chart serving every environment, with per-environment values files supplying only the differences.
- **Kustomize** — a common base plus small per-environment overlay patches, written in plain Kubernetes YAML.
- **Rendered Output** — this third box is drawn explicitly because it is the thing people most often forget exists. Kubernetes never sees a Helm chart or a Kustomize overlay. It only ever sees ordinary manifests. The chart and the overlay are conveniences for humans; this box is the actual contract with the cluster.

Layer 7 — The Kubernetes clusters

The three clusters are drawn side by side because they are peers receiving the same artifact, not stages in a pipeline that transform it. What differs between them is written on each box: development syncs automatically with self-heal on; staging mirrors production and runs integration tests; production syncs after approval and rolls out progressively. The application image itself is identical in all three.

The arrow leaving this layer ("the live state is watched continuously") is what distinguishes this architecture from a traditional pipeline. A pipeline finishes. This does not — it hands over to Figure 4 and keeps running for the lifetime of the service.

5.5 Figure 4 — The Reconciliation and Drift-Detection Loop

Figure 4 - The Reconciliation and Drift-Detection Loop

This loop runs by itself every few minutes, forever, for every application in every cluster.

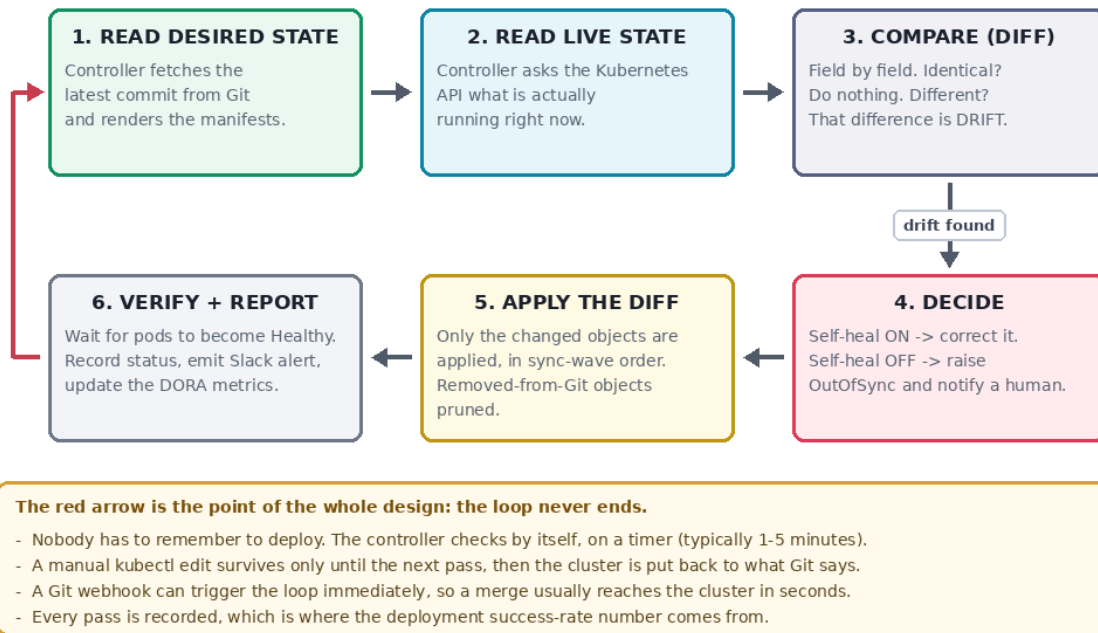


Figure 4: layer 8. The red arrow returning from step 6 to step 1 is the most important line in the entire architecture.

Each step of the loop

Step	What happens	Why it is necessary
1. Read desired state	Fetch the latest commit from Git and render the manifests.	Establishes the authoritative answer to "what should be running?" from a source that cannot be quietly edited.
2. Read live state	Ask the Kubernetes API what is actually running right now.	Establishes the truth about reality, rather than assuming the last deployment is still intact.
3. Compare (diff)	Compare field by field. Identical means do nothing; different means drift.	Applying only the difference is what makes the loop cheap enough to run every minute, forever.
4. Decide	Self-heal on: correct it. Self-heal off: raise OutOfSync and notify.	The right answer differs by environment. Automatically undoing an engineer's emergency change mid-incident in production would be harmful.
5. Apply the diff	Apply only the changed objects, in sync-wave order, and prune anything removed from Git.	Ordering prevents dependency failures; pruning is what keeps Git a <i>*complete*</i> description rather than an additive one.

Step	What happens	Why it is necessary
6. Verify and report	Wait for healthy status, record the outcome, notify, update metrics.	Turns "the command succeeded" into "the software is working", which is the only definition of success worth measuring.

The red return arrow from step 6 back to step 1 is what makes this architecture different in kind from a deployment pipeline. A pipeline runs when triggered and then forgets. This loop never stops, which means the system defends its own correctness with no human involvement. Drift detection, self-healing, disaster recovery and "the cluster cannot silently rot" are all just consequences of that one arrow existing.

5.6 Figure 5 — Environment Promotion

Figure 5 - Environment Promotion

The same artifact moves forward. The image is built once; promotion is only a Git change that points the next environment at it.

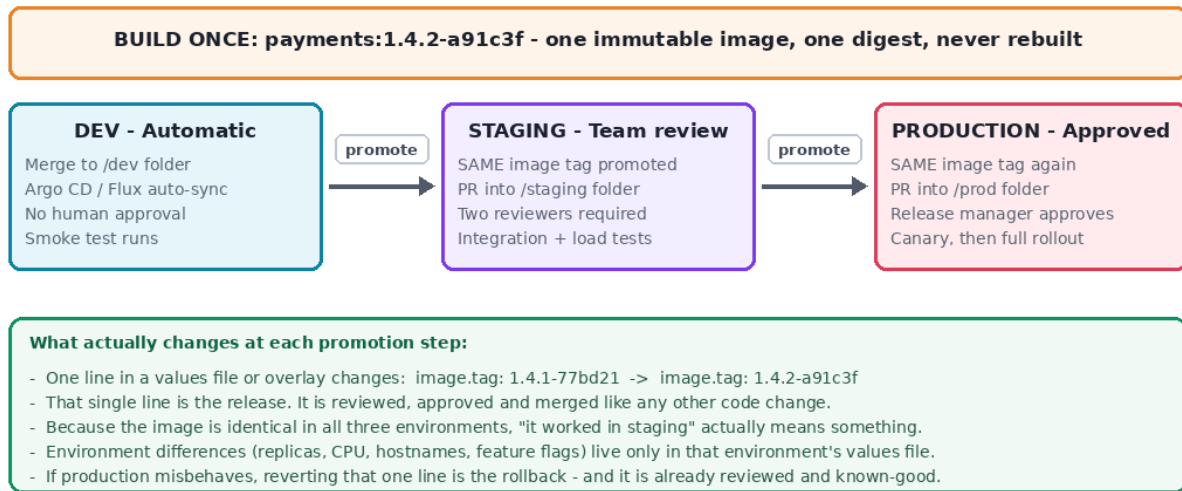


Figure 5: the same artifact moves forward. Promotion changes a pointer in Git, never the software itself.

The orange bar across the top is deliberately drawn as a single wide box spanning all three environments, because that is exactly what it represents: **one** image, built **once**. The three environment boxes below it are consumers of that one artifact, not producers of their own.

Each "promote" arrow represents a pull request that changes one line — an image tag — in the next environment's values file or overlay. That one line is the release. Everything else about promotion (approval, review, audit, rollback) is inherited free from Git, because the release is an ordinary reviewable commit.

What differs between the three boxes is only governance and shape: who approves, how many replicas, which hostname, and whether the rollout is straight or canary. Those differences live in that environment's values file, so they are visible, reviewable and diffable — never hidden in a script or in somebody's memory.

5.7 Figure 6 — Supporting Services

Figure 6 - Supporting Services Used by Every Layer

These run alongside the platform and are themselves managed by GitOps.

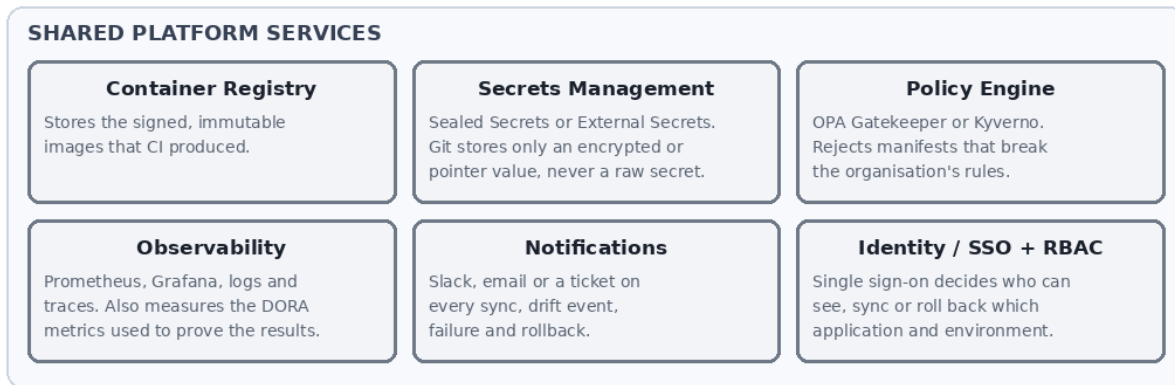


Figure 6: layer 9. These services are used by every layer above, and are themselves managed by GitOps.

This layer is drawn last but is not optional. The registry stores the artifacts the whole flow depends on; secrets management is what makes "everything in Git" possible without committing passwords; the policy engine is the last line of defence at the cluster boundary; observability is what produces the metrics quoted in the summary; notifications close the loop back to humans; and identity decides who may do what.

One detail matters more than it first appears: **these services are themselves deployed by GitOps**, from the infrastructure repository. The platform manages itself by the same rules it imposes on everyone else, which is what stops the platform from becoming the one hand-maintained, undocumented thing in the organisation.

5.8 Figure 7 — One-Click Rollback

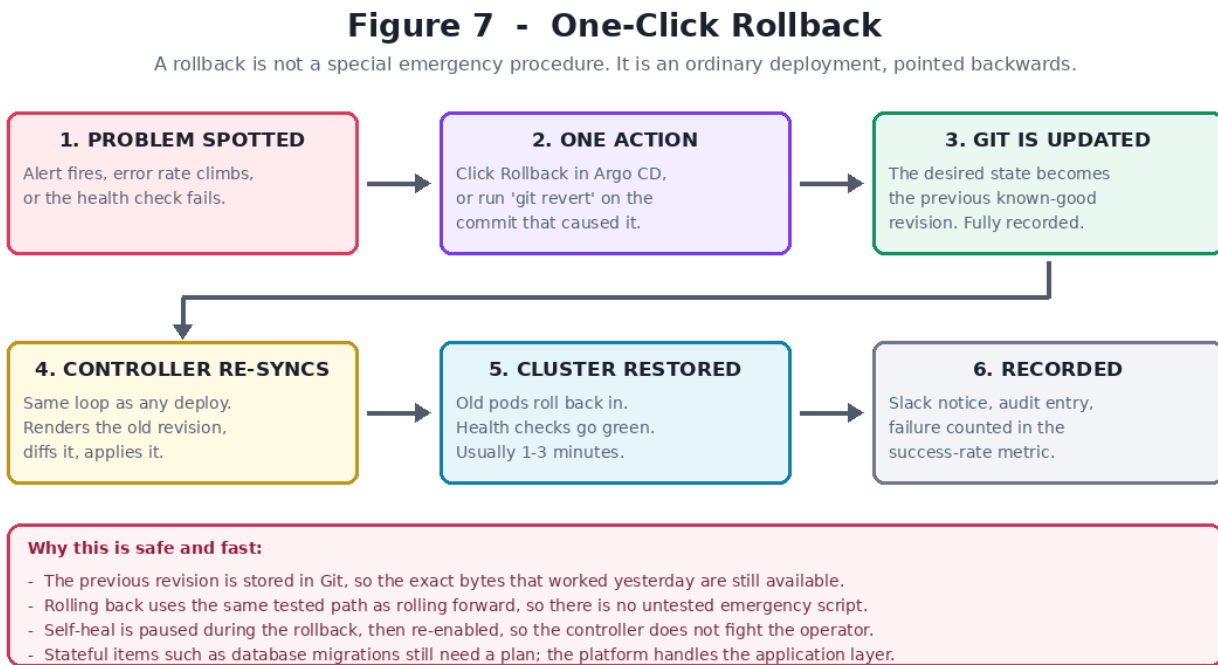


Figure 7: rollback is an ordinary deployment pointed backwards, not a special emergency procedure.

The single most important thing to notice in Figure 7 is that steps 4, 5 and 6 are **identical to a normal deployment**. There is no separate emergency mechanism. This is deliberate: an emergency procedure that is only used during emergencies is, by definition, the least-tested procedure you own. Here, the rollback path is exercised by every ordinary release, hundreds of times, which is why it can be trusted at the worst possible moment.

Step 2 offers the two routes described earlier. Reverting in Git is preferred because Git remains truthful and self-heal will not fight the change. Pressing Rollback in Argo CD is faster and is used when seconds matter — with the mandatory follow-up of reverting Git afterwards, or the next reconciliation pass will faithfully reapply the broken version.

5.9 Every Arrow in the Architecture, in One Table

From	To	What travels along it	Direction matters because
People	Self-service platform	An intention, in plain words	It is the only entry point. There is no side door.
Self-service platform	Git	A pull request containing generated files	The portal produces a proposal, never a deployment.
App repo	CI pipeline	A merged commit	Building is triggered by code changes only.
CI pipeline	Registry	A signed, immutably tagged image	The artifact is created once and never again.

From	To	What travels along it	Direction matters because
CI pipeline	Git	A one-line image-tag pull request	CI has no cluster credentials. This is its maximum power.
Git	Controllers	Notification of a new commit (webhook or poll)	This is a notification, not a command. The cluster decides when to act.
Controllers	Git	A fetch of the repository at a commit	Outbound only. The cluster API never needs to be exposed.
Controllers	Rendering engine	Chart, values and overlay paths	Rendering happens in-cluster, so it is identical every time.
Rendering engine	Kubernetes API	Plain manifests, applied as a diff	Only differences are applied, keeping the blast radius minimal.
Kubernetes API	Controllers	The live state of every managed object	This return path is what makes drift detection possible at all.
Controllers	Notifications	Sync, drift, failure and rollback events	Humans are informed by the system rather than having to watch it.
Registry	Kubernetes nodes	The image being pulled at start-up	The identical image in every environment.
Git	Controllers (rollback)	The previous known-good revision	Rollback reuses the ordinary delivery path, so it is well tested.

6. How the Results Were Achieved and Measured

6.1 Where the 55% Lead-Time Reduction Came From

Lead time is measured objectively: the elapsed time from the first commit of a change to that change running healthily in production. Both endpoints come from timestamps the system records anyway — the Git commit and the controller's healthy event — so the figure is calculated, not estimated.

Change made	Time removed	Why
Removed the human queue	Largest	Deployment used to wait for a specific person to be available. A merge now triggers the work immediately, at any hour.
Removed manual configuration work	Large	Configuration is generated from templates and updated by a bot, instead of being hand-written for each environment.
Replaced manual review with automated checks	Large	Validation that took fifteen minutes of careful reading takes two minutes of automation, and catches more.
Removed repeated work per environment	Large	The same artifact is promoted with a one-line change instead of the deployment being performed again from scratch.
Made rollback cheap	Indirect but large	Cheap rollback allowed small, frequent releases instead of large batched change windows, which shortened lead time again.
Webhook-triggered sync	Small but visible	The delay between merge and the cluster starting work fell from minutes to seconds.

6.2 Where the Above-99% Success Rate Came From

Success rate is measured as the proportion of production syncs that reached a Healthy state without requiring a rollback or manual intervention. It rose because a bad change now has to survive eight independent barriers to reach a user:

1. Unit and integration tests in CI.
2. Dependency and container image scanning.
3. Schema and template validation on the configuration change.
4. Policy checks enforcing organisational rules.
5. A rendered diff preview reviewed by a human before merge.
6. A server-side dry run performed by the controller before applying.
7. Sync waves and PreSync hooks that stop a rollout when a dependency fails.
8. Health checks, canary analysis and automatic rollback after applying.

Two structural changes matter more than any individual check. First, **environments stopped diverging**, because drift is now detected and corrected continuously — so a successful staging test genuinely predicts a successful production release. Second, **deployment stopped being a human performance**. The same automation runs the same way at 2am on a Sunday as at 11am on a Tuesday, which removes the entire category of failures caused by a tired person skipping or repeating a step.

6.3 The Metrics That Are Tracked

Metric	Definition	Where the data comes from
Deployment frequency	How often a change reaches production.	Count of successful production syncs.
Lead time for change	First commit to healthy in production.	Git commit timestamp and the controller's healthy event.
Change failure rate	Share of production releases needing a rollback or urgent fix.	Rollback events and incidents linked to a release.
Time to restore service	Detection of a problem to service restored.	Alert timestamp and the healthy event after rollback.
Drift events	How often live state departed from Git.	OutOfSync events from Argo CD and Flux.
Sync success rate	Share of all syncs that reached Healthy.	Controller metrics.

7. Design Decisions and Honest Trade-Offs

These are the choices that shaped the architecture, and what each one cost.

Decision	Why it was made	What it costs
Pull model instead of push	No external system holds cluster credentials; the cluster API can stay private; drift is detected continuously.	Slightly less immediate than a push, and mitigated with webhooks. Debugging is less familiar to engineers used to pipelines.
Separate application and configuration repositories	A configuration change does not require rebuilding the application; different approval rules per repository; no pipeline loop.	Two repositories to keep in step, and one extra hop for a developer to understand.
Both Argo CD and Flux	Argo CD gives developers visibility and one-click rollback; Flux gives unattended infrastructure automation and image updates.	Two systems to operate, upgrade and learn. Requires strict ownership boundaries to prevent them conflicting.
Both Helm and Kustomize	Helm consumes vendor charts without forking them; Kustomize keeps our own services readable as plain YAML.	Two configuration idioms in one repository, which needs clear documented conventions.
Auto-heal on in lower environments, alert-first in production	Correctness matters most in dev and staging; during a production incident, undoing an engineer's emergency change automatically would be dangerous.	Production can stay OutOfSync longer than ideal, so the alert has to be treated seriously.
Immutable, commit-stamped image tags	A description in Git identifies exactly one set of bytes forever, which is the foundation of promotion and rollback.	More tags to store and a retention policy to manage.
Self-service portal as the front door	Removes the platform team as a queue and guarantees every new service starts correct.	The portal and templates are themselves a product that must be maintained and kept current.

7.1 What This Architecture Does Not Solve

- **Database schema changes.** Reverting a Git commit does not un-drop a column. This needs the expand-and-contract migration discipline described in section 3.13.
- **A bad change that is correctly deployed.** If the code is wrong but healthy, the platform will deploy it perfectly. Testing remains the answer.
- **Stateful data recovery.** Rollback restores the application, not the data. Backups remain a separate concern.
- **Culture.** If engineers keep making manual cluster changes, they will see constant drift alerts. The platform makes the right path easier, but it does not remove the need for the team to agree on it.

8. Glossary

Term	Plain-English meaning
Argo CD	A GitOps controller with a web interface, used here for applications, visual diffs and one-click rollback.
Artifact	The built, packaged output of a build — here, a container image.
Base (Kustomize)	The common set of manifests shared by every environment.
Canary	Sending a small share of live traffic to a new version first, and watching the metrics before continuing.
Chart (Helm)	A packaged, reusable set of Kubernetes templates for one application.
CODEOWNERS	A file saying which people or teams must approve changes to which paths.
Commit	One recorded change in Git, with an author, a time and a message.
Container image	A sealed package containing an application and everything it needs to run.
Declarative	Describing the desired end result rather than the steps to reach it.
Desired state	What the system should look like. Lives in Git.
Digest	A cryptographic hash identifying an image's exact contents; cannot be reused for different bytes.
DORA metrics	Four standard delivery measures: deployment frequency, lead time, change failure rate, time to restore.
Drift	A difference between the live state and the desired state.
Flux	A GitOps controller built as several focused controllers, used here for infrastructure and image automation.
GitOps	Using Git as the single source of truth, with automation continuously enforcing it.
Health check	The controller's assessment of whether a deployed resource is actually working, not merely accepted.
Helm	A templating and packaging tool for Kubernetes configuration.
HelmRelease	A Flux object describing which chart, version and values should be installed.
Hook	A job run before, during or after a sync — commonly a migration or a smoke test.
Idempotent	Safe to run repeatedly; running it again changes nothing.

Term	Plain-English meaning
Immutable tag	An image tag that can never be overwritten once published.
Ingress	The rule allowing outside traffic to reach a service by hostname and path.
Kubernetes	The system that runs and supervises containerised applications across many machines.
Kustomize	A tool that layers small environment-specific patches over a common base of plain YAML.
Lead time	The elapsed time from a change being written to it running in production.
Live state	What is actually running right now, read from the Kubernetes API.
Manifest	A YAML file describing a Kubernetes object.
Namespace	A folder inside a cluster used to separate teams and environments.
OutOfSync	The status meaning the live state no longer matches Git.
Overlay (Kustomize)	The environment-specific patches applied on top of a base.
Pod	One running instance of an application inside Kubernetes.
Policy engine	Software (OPA Gatekeeper, Kyverno) that rejects configuration breaking organisational rules.
Promotion	Moving a tested version forward from one environment to the next.
Prune	Deleting cluster objects that have been removed from Git.
Pull model	The cluster fetches changes itself, rather than being pushed to from outside.
Pull request	A proposed change, opened for review before merging.
Readiness probe	A check that decides whether a pod is ready to receive traffic.
Reconciliation	The repeating loop that makes the live state match the desired state.
Registry	The storage system holding container images.
Rollback	Returning to the previous working version.
Rolling update	Replacing old pods with new ones gradually, so the service stays available.
Self-heal	Automatically correcting drift without waiting for a human.
Sync	Applying the desired state from Git to the cluster.
Sync wave	An ordering number controlling which resources are applied before which others.
Values file (Helm)	The per-environment settings that fill in a chart's placeholders.

Term	Plain-English meaning
Webhook	A message from Git telling the controller that something changed, so it can react immediately.

9. Questions People Usually Ask About This Project

Why not just use a normal CI/CD pipeline to deploy?

A pipeline deploys and then forgets. It has no idea whether the cluster still matches what it deployed an hour later. It also needs powerful cluster credentials, which is a standing security risk. GitOps keeps checking forever and needs no inbound access to the cluster.

What stops someone bypassing all of this with kubectl?

Three things. Cluster RBAC restricts who can write directly at all. Admission policies reject non-compliant objects regardless of how they arrive. And reconciliation reverts unauthorised changes on the next pass, so a manual edit does not survive. In production the change is alerted rather than silently reverted, so a legitimate emergency fix is not undone mid-incident.

Isn't running both Argo CD and Flux redundant?

They serve different users. Argo CD gives developers and release managers visibility, an approval workflow and a rollback button. Flux gives platform engineers unattended automation, including writing new image tags back into Git. They are given strictly separate ownership by repository path so they never manage the same resource.

What happens if Git is unavailable?

Running applications are unaffected — they keep serving traffic. New deployments pause until Git returns. The controllers retry, and reconciliation resumes automatically. This is a deliberately safe failure mode: the system stops changing rather than starts guessing.

How do you deploy something urgently at 3am?

The same way as at 3pm: open a pull request, get the required approval, and merge. That takes minutes, and it is faster than the old manual process ever was. If an immediate manual intervention is genuinely needed, an engineer can make it, and production alerts on the resulting drift so that the change is written back into Git afterwards rather than being forgotten.

What was the hardest part?

Not the tooling. The hard parts were the boundaries: deciding which controller owns which resources so the two never fight; tuning diff normalisation so that fields the cluster manages itself do not register as permanent false drift; and getting teams comfortable with the idea that a manual cluster change will simply disappear. The technology is well documented; the ownership model and the habits are what has to be designed carefully.